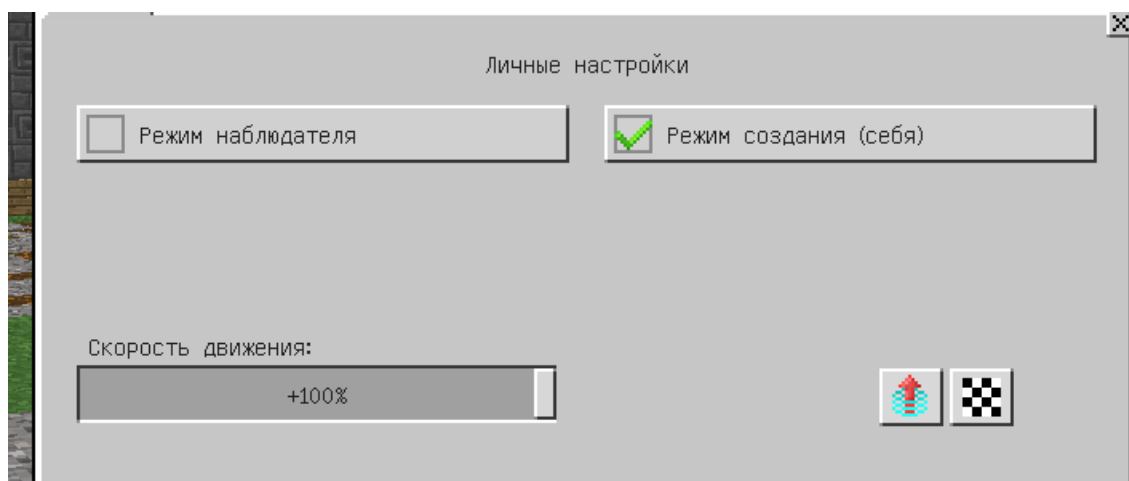


# Урок 5

|                     |                                                                                                                                                                                   |
|---------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Курс                | Знакомство с Minecraft EDU. Основные способы взаимодействия с «черепахой – роботом»                                                                                               |
| Тема урока          | <ul style="list-style-type: none"><li>· знакомство с компьютерами;</li><li>· взаимодействие с командной строкой;</li><li>· написание текстовых программ</li></ul>                 |
| Цель урока          | <ul style="list-style-type: none"><li>· научиться работать с командной строкой компьютера;</li><li>· научиться писать текстовые программы</li></ul>                               |
| Мотивация           | Вы научитесь программировать по-настоящему                                                                                                                                        |
| Средства            | Компьютерный класс, Minecraft EDU                                                                                                                                                 |
| Задачи              | <ul style="list-style-type: none"><li>· знакомство с командной строкой компьютера;</li><li>· создание каталогов и файлов;</li><li>· написание программ с помощью текста</li></ul> |
| Ожидаемый результат | Усвоение навыков работы с компьютером и написанием программ.                                                                                                                      |

## Руководство для преподавателя

1. Чтобы быстро перемещаться по серверу, нажмите M (английскую) и выставьте для себя следующие настройки:



2. Если необходимо быстро переместиться к ученику, нажимаете T (английскую) и вводите команду /tp (ник ученика), после чего нажимаете Enter.

3. Если ученик где-то застрял, потерялся, вы пишете /tp (ник ученика) (ваш ник) и отправляете его на локацию, номер которой он проходил.

4. Каждая станция состоит из пяти отсеков, каждый отсек – это отдельный урок.

5. Если ученик справился с заданием на урок раньше, то его можно отправить в телепорт с названием Free, где он сможет строить, что угодно, или выполнять дополнительное задание.

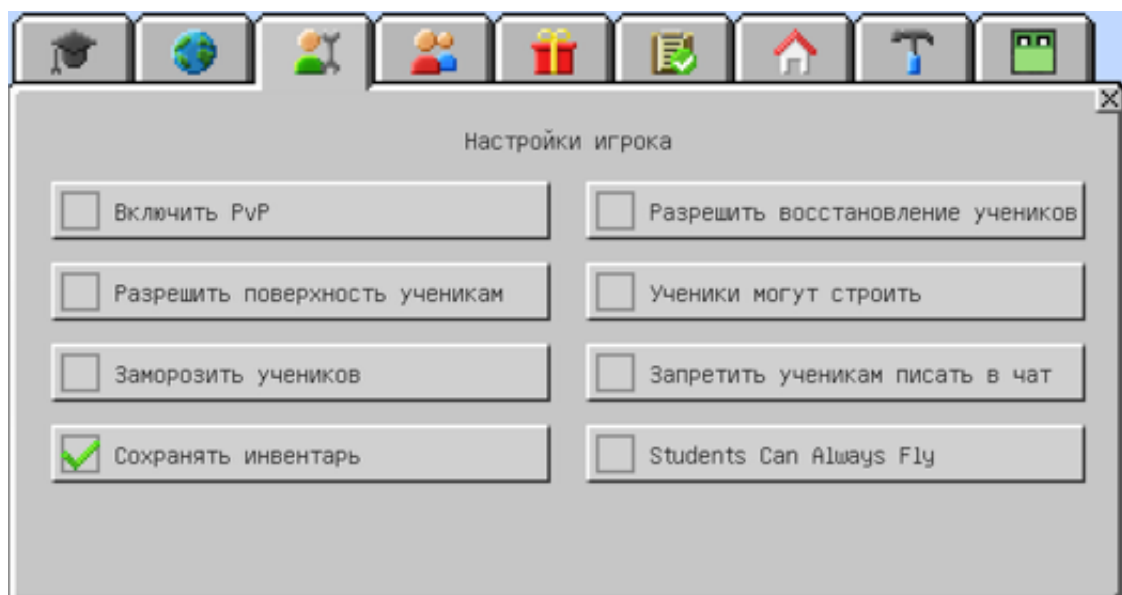
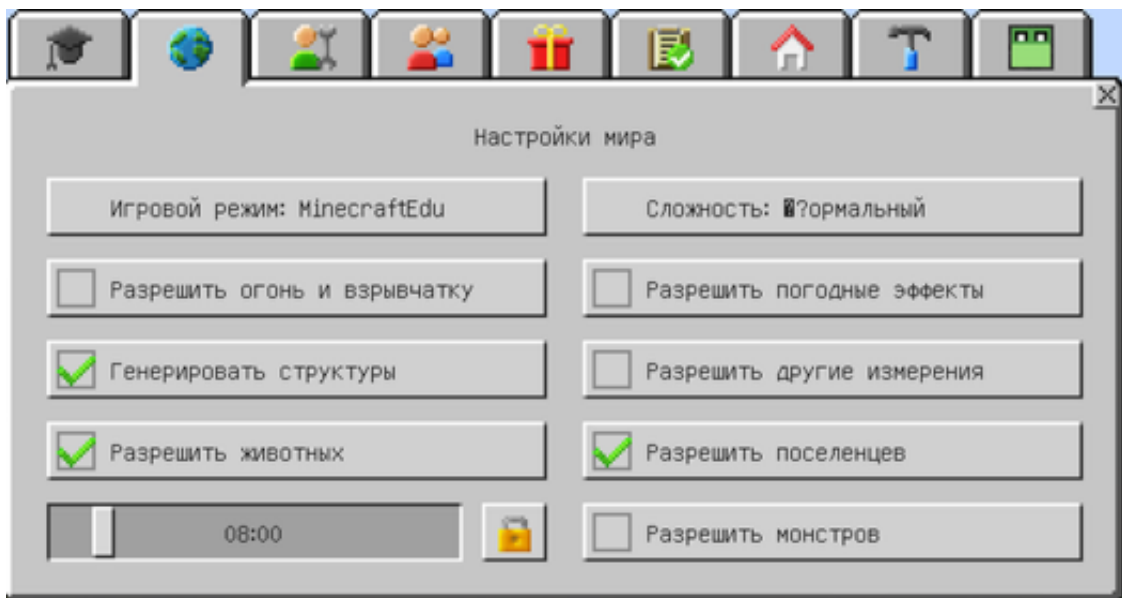
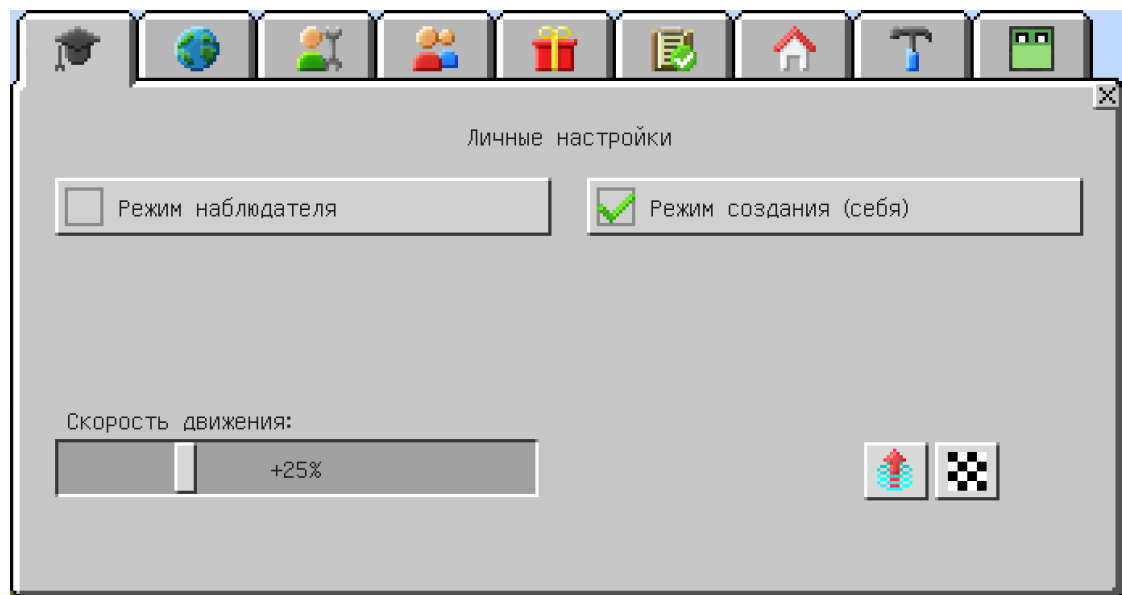
6. Лабиринт необходимо проходить только на первом уроке, в течение последующих уроков необходимо воспользоваться блоком телепорта на спавне в лабиринте. После этого каждый ученик должен пробежать до необходимого отсека, игнорируя их задания.

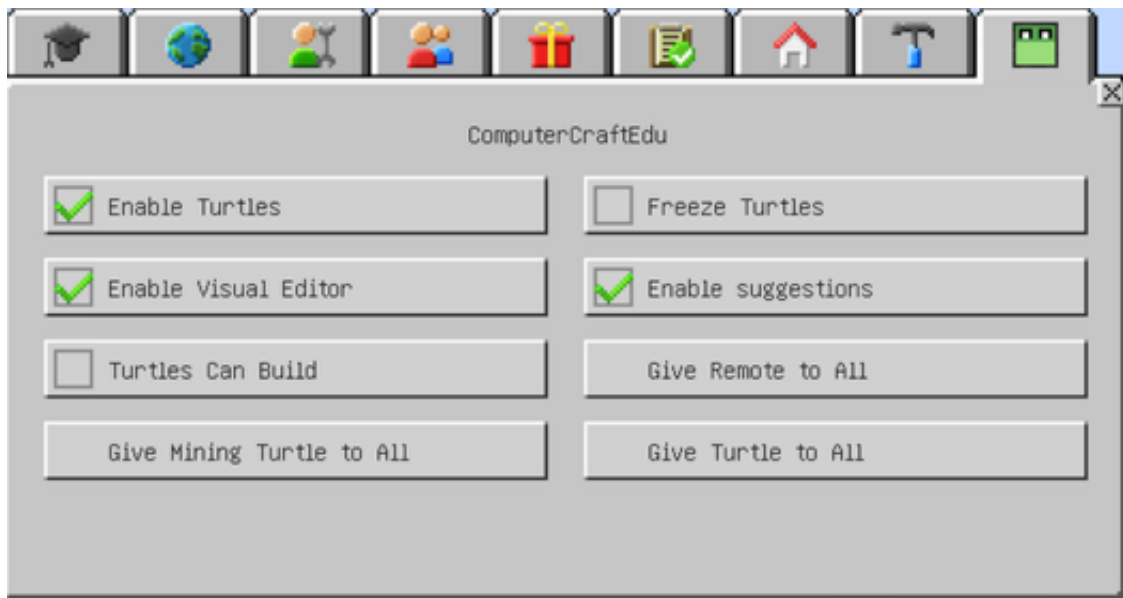
7. Каждый ученик должен иметь уникальный никнейм, в ином случае другой ученик с таким же именем будет появляться на том же месте.

8. На блоке с зеленым кругом ученик может сломать черепашку киркой, если это потребуется.

9. Каждый ученик проходит свою цепочку квестов, проходить квест с кем-либо нельзя.

10. Обязательно выставить рекомендуемые настройки мира (нажатие кнопки M):





11. Для того чтобы летать, необходимо быстро нажать два раза клавишу ПРОБЕЛ.

12. Также рекомендуется выключить музыку в игре. Для этого необходимо нажать клавишу ESC – Настройки – Музыка и звуки – ползунок «музыка» переместить в позицию ВЫКЛ.

13. Код ученика может не совпадать с рекомендуемым. Главное – чтобы был соблюден принцип его написания, исходя из темы урока, и результат в виде постройки.

14. В начале урока всем обязательно необходимо взять пульт управления черепашкой, кирку и черепашку. Они находятся у телепорта Lobby, на скрине обозначены соответствующими буквами:







– так выглядит пульт. Необходимо взять его в руку и правой кнопкой мыши нажать на свою черепашку, после этого с ней можно работать.

## **Методический план урока**

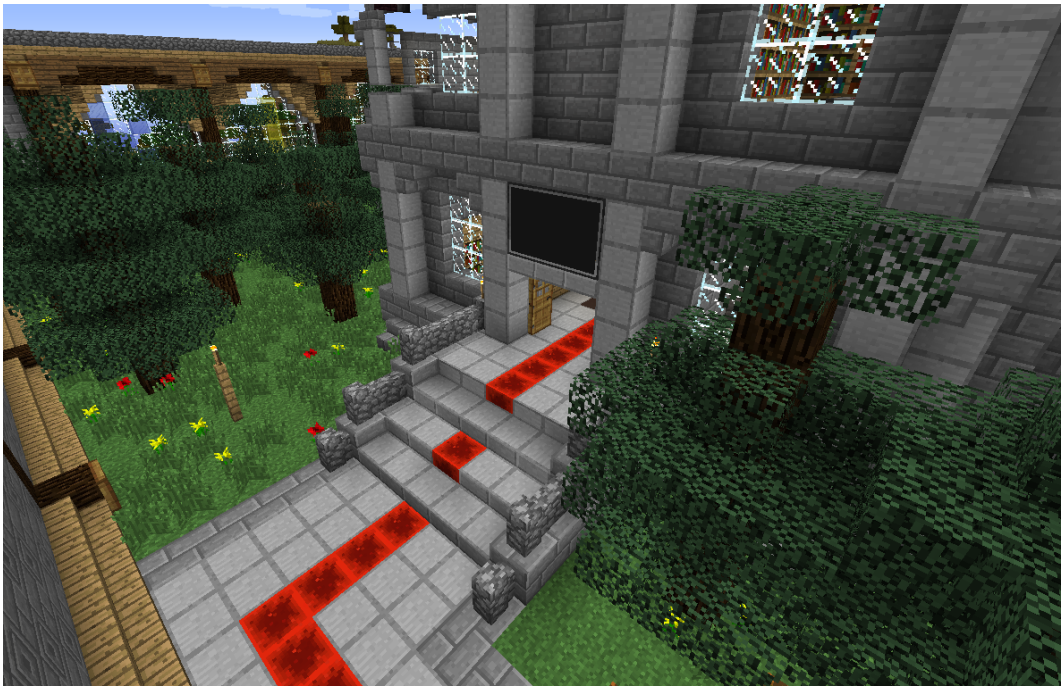
**Организационный момент – 10 мин.**

**Теоретическая часть – 30 мин.**

**Практическая часть – 80 мин.**

Приветствую вас на 5 уроке. Сегодня мы познакомимся с возможностями компьютера в гостях у Техномага.

Ученики заходят на территорию 5 урока и замечают большую библиотеку, на стенах которой установлены дисплеи. Ученики идут по тропинке ко входу в библиотеку:



Сразу на входе их встречает табличка, которая направляет их в кабинет Техномага:



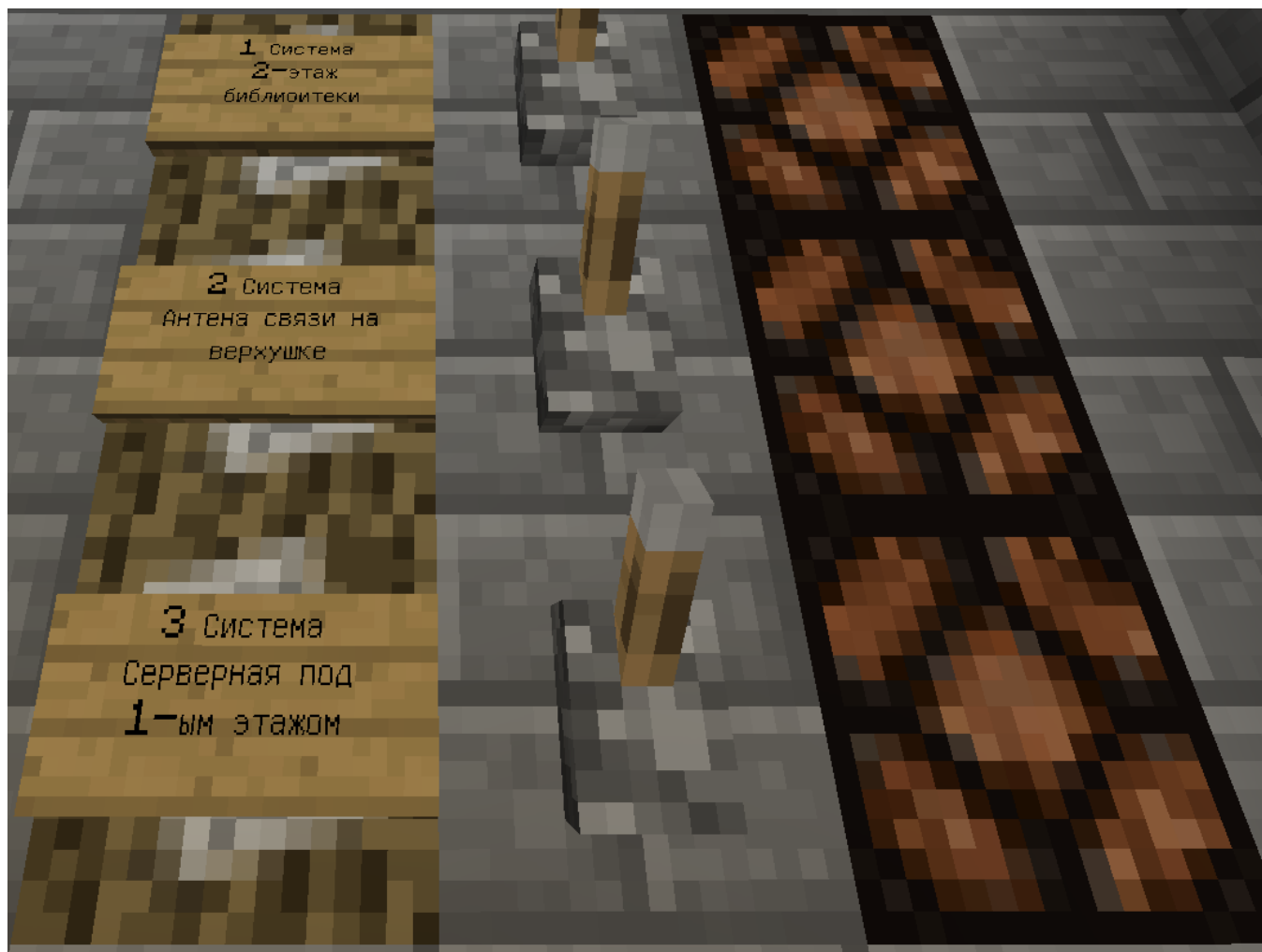
Ученик читает 2 страницы из информационного блока на рабочем столе у Техномага. Он просит ученика помочь ему с восстановлением программ его библиотеки после кибератаки:



Для этого ученик должен выполнить 3 задания (обозначены как системы) на стене у входа. В каждом информационном блоке содержится информация о локации в библиотеке, в которую ученик должен отправиться, а также информация о том, что на локации задания в

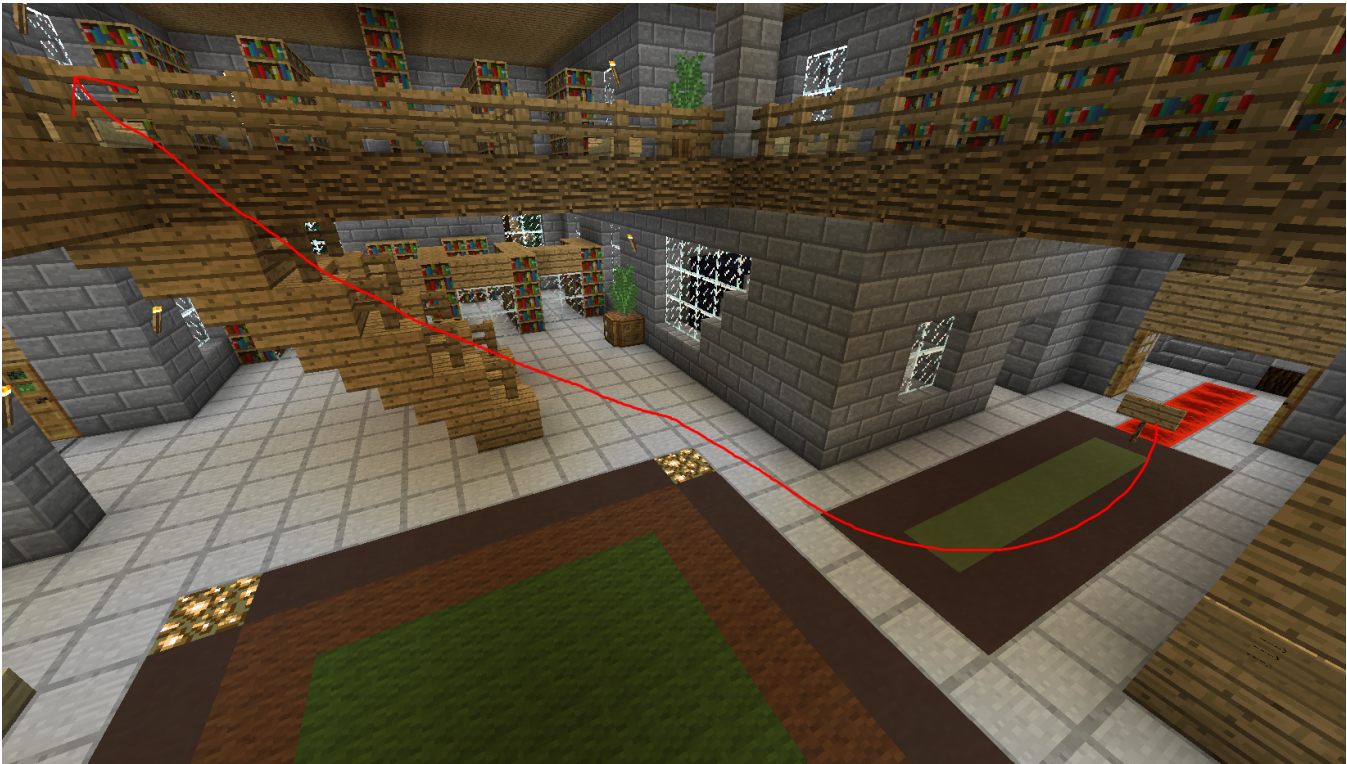


сундуке ученика будут ждать инструкции, необходимые для выполнения задания:



Ученик направляется на локацию и берет инструкции из сундука. В этих инструкциях содержится вводная информация о том, что ученику необходимо сделать, а также теория и практика для выполнения задания.

**Система 1. Задание:** Отправляйся на 2-ой этаж библиотеки. Там ты увидишь серверную библиотеки и стол. Рядом со столом ты увидишь сундук с инструкциями, которые подскажут тебе, что делать:





**Система 1. Введение:** Злой техномаг стер каталоги книг! Я прошу тебя помочь восстановить их.

Тебе необходимо будет восстановить 5 каталогов: Fantasy, Child, Detective, Science, History.

Чтобы это сделать, тебе необходимо вводить команды на компьютере. Для этого нажми правой кнопкой мыши по золотистому компьютеру на столе. Тебе откроется командная строка.

Ты увидишь надпись CraftOS 1.7, а на 2-ой строке – знак «>» и мигающий курсор. На строке с мигающим курсором ты будешь писать команды. В компьютере находятся каталоги, по которым можно перемещаться, и файлы, которые ты можешь открывать и редактировать.



Пустота перед знаком «>» означает, что ты находишься в корневом каталоге. А, например, строка Books> означает, что ты находишься в каталоге Books.

**Система 1. Команды:** Тебе понадобятся следующие команды:

1. `dir` – показывает каталоги (зеленые) и файлы (белые) в текущем каталоге;
2. `mkdir [название каталога]` – создает новый каталог в текущем каталоге;
3. `cd [путь к каталогу]` – перемещается к указанному каталогу.

Пример. Создадим каталог Fantasy.

1. Напиши свою первую команду `dir`. Ты увидишь каталог `rom`. Это системный каталог, который, к счастью, не поврежден
2. Теперь напиши команду `mkdir Fantasy`. Поздравляю, ты создал свой первый каталог!
3. Напиши команду `cd Fantasy`. Ты переместишься в только что созданный каталог.
4. Вернись назад с помощью команды `cd ../`.
5. Введи команду `dir`. Ты увидишь каталоги `Fantasy` и `rom`.

Отлично! Тебе осталось создать оставшиеся 4 каталога.

В итоге должно получиться следующее:

```
CraftOS 1.7
> dir
rom
> mkdir Fantasy
> cd Fantasy
Fantasy> cd ../
> dir
Fantasy rom
> mkdir Child
> mkdir Detective
> mkdir Science
> mkdir History
> mkdir
Usage: mkdir <path>
> dir
Child      Detective Fantasy  History   Science
rom
> _
```

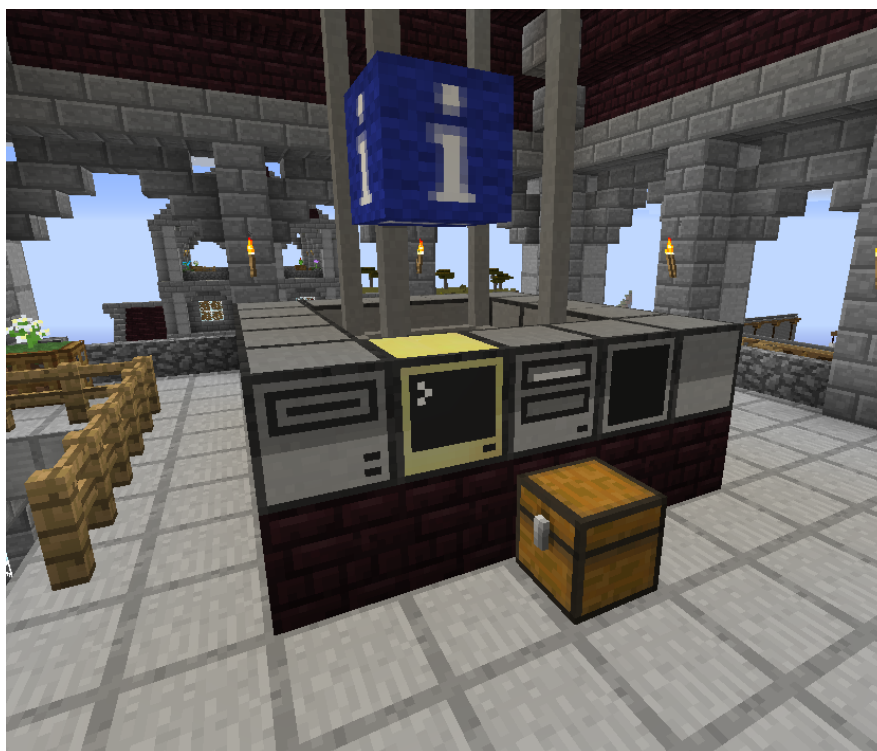
После выполнения задания ученик отправляется обратно к стене с заданиями, чтобы отметить прогресс. И так после каждого выполненного задания. После того, как ученик переключит рычаг 3-его задания, активируется механизм, который даст ученику книгу с благодарностью и новое изобретение Техномага – смартфон:





**Система 2. Задание:** Отправляйся на самый верх. Тебя будет ожидать пункт связи. Рядом с пунктом связи ты увидишь сундук с инструкциями, которые подскажут тебе, что делать:





**Система 2. Введение:** С компьютера на этом пункте связи была стерта система авторизации. Из-за этого никто не может войти в систему! Ты должен помочь написать систему авторизации. Сейчас расскажу, как это сделать.

Система входа пользователей в систему состоит из 3-ех этапов:

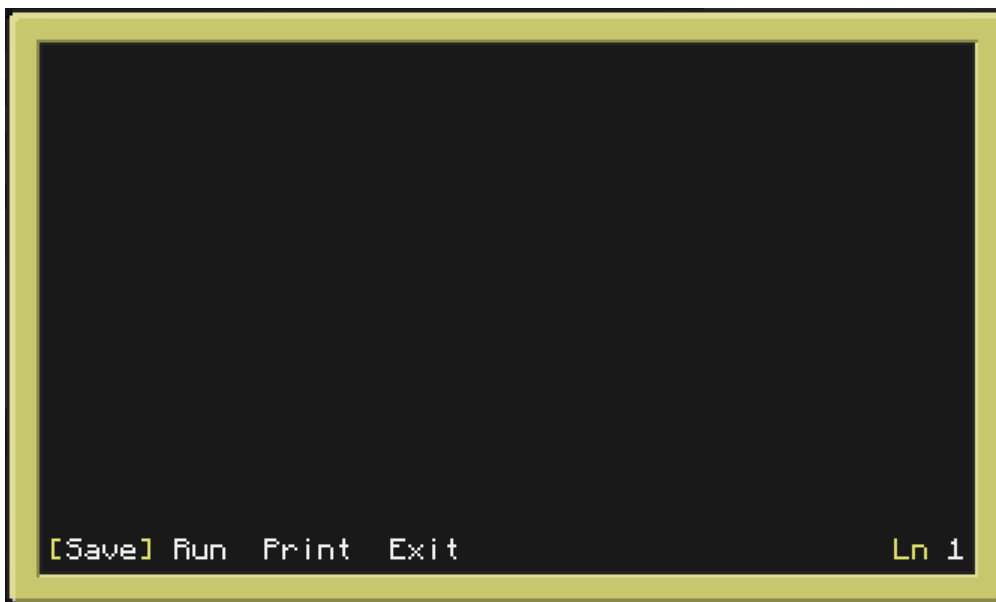
1. идентификация – процесс установления личности пользователя.  
Проще говоря, проверка логина (например, имени пользователя);
2. аутентификация – проверка подлинности того, кто хочет получить доступ к аккаунту. То есть проверка введенного пароля;
3. авторизация – определяет полномочия и права доступа у вошедшего пользователя.

Тебе предстоит написать программу, которая проверяет логин и пароль для входа в систему. Авторизацией я займусь позже.

Программы пишутся в файлах, а файлы находятся в папках (то же самое, что каталоги). Первым делом тебе необходимо научиться создавать файлы. Для создания файла тебе нужна команда `edit [название файла]`. Попробуй ввести команду `edit file1`. Таким образом, ты создашь файл и окажешься в редакторе файла. В строке с мигающим курсором ты пишешь текст или команды. Снизу ты увидишь надпись «Press Ctrl to access menu». Нажав на клавиатуре левую кнопку Ctrl, ты увидишь кнопки:

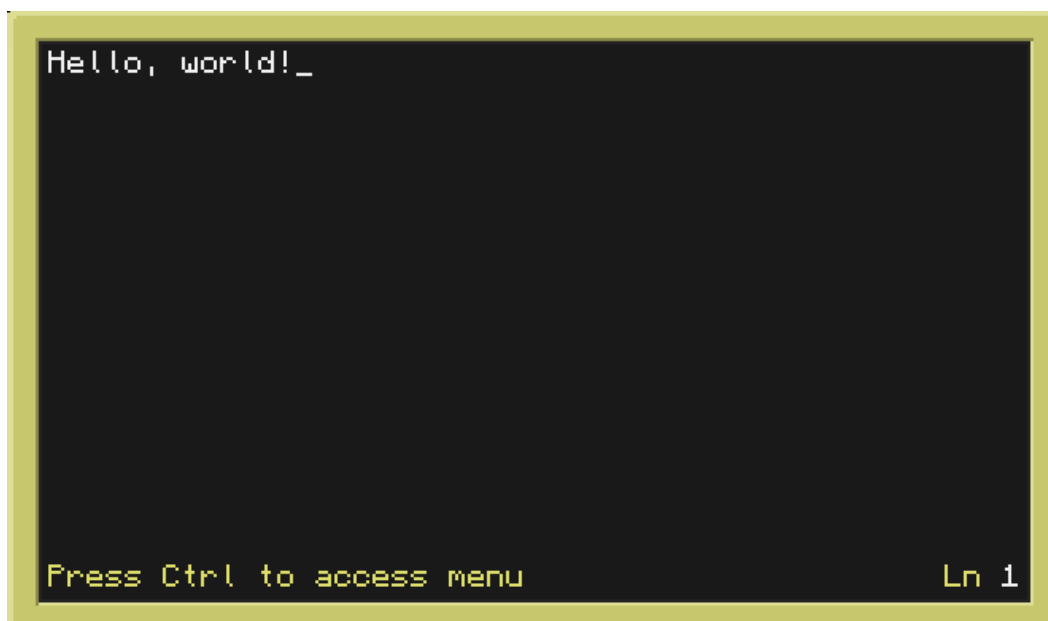
1. Save – сохранить файл;
2. Run – запустить программу;
3. Print – распечатать файл на бумаге с помощью принтера;
4. Exit – выход из редактирования файла в командную строку.

Чтобы вернуться к редактированию файла, снова нажми левый Ctrl:



Поработай с файлом:

1. напиши текст на английском языке «Hello, world!»;
2. затем сохрани файл;
3. теперь выйди из файла в командную строку;
4. просмотри папки и файлы знакомой тебе уже командой `dir`;
5. снова открой файл, написав команду `edit file1`. Эта команда не только создает файл, но и открывает их:





Здорово, теперь ты умеешь создавать и редактировать файлы! Чтобы написать в файле программу входа в систему, читай книги «Программирование».

## Система 2. Программирование 1.

Для программирования этой системы тебе понадобятся следующие команды:

1. `write()` и `print()`. Эти команды выводят информацию на экран. В круглых скобках указываются аргументы (текст в кавычках, который будет выведен на экран). Достань из сундука распечатанную страницу «Пример 1». Там ты увидишь код. Создай на компьютере файл «prog1» и напиши в этом файле код из страницы «Пример 1»:

```
write("write: Hello, ")  
write("world")  
  
print(" ")  
print(" ")  
  
print("print: Hello, ")  
print("world")
```

Далее сохрани файл и запусти программу, нажав кнопку Run. Ты увидишь результат работы программы:

write: Hello, world

print: Hello,

world



Разница между print() и write () в том, что 1-ая команда переводит курсор на следующую строку, а 2-ая – оставляет курсор на своей строке:

```
write("write: Hello, ")
write("world")

print(" ")
print(" ")

print("print: Hello, ")
print("world")_
```

```
edit  shell
write: Hello, world

print: Hello,
world
Press any key to continue
```

После запуска программы ты окажешься во вкладке «shell», в которой происходит выполнение программы. Чтобы вернуться к редактированию файла, наведи курсор мыши на вкладку «edit» и нажми на нее левой кнопкой мыши.

## **Система 2. Программирование 2. Создание переменных.**

В переменных можно хранить информацию, которая может пригодиться в процессе выполнения программы. Например, в переменной username хранится имя пользователя. Команда var1=123 создает переменную var1, в которой хранится число 123, а команда var2="i'm cool" создает переменную var2, в которой хранится строка "i'm cool". В основном в переменных можно хранить три типа данных:

1. числа;
2. строки. Они должны быть в кавычках;
3. булевый тип данных. Это логический тип данных. Переменная с этим типом данных хранит значение true (правда) или false (ложь).

Этот тип данных пригодится тебе в дальнейшем.

Попрактикуйся в написании переменных. В сундуке ты найдешь страницу «Пример 2»:

```
var1 = 54
var2 = "minecraft"
var3 = true
var4 = false

print(var1)
print(var2)
print(var3)
print(var4)
```

Создай файл prog2 и перепиши в него код из 2-го примера, но названия и значения переменным ты можешь дать сам. Если в аргументах команды print() или write() написать название переменной, то ее значение выведется на экран. Попробуй запустить программу:

```
var1 = 54
var2 = "minecraft"
var3 = true
var4 = false

print(var1)
print(var2)
print(var3)
print(var4)_
```

```
edit  shell
54
minecraft
true
false
Press any key to continue
```

## Система 2. Программирование 3. read().

Эта команда позволяет получить данные от пользователя. Доходя до этой команды, программа останавливается и ждет, пока пользователь введет текст или число. После этого пользователь нажимает Enter на клавиатуре. Полученное число или текст можно сохранить в

переменную. Давай попробуем:

1. создай файл prog3;
2. в первой строке напиши команду name=read();
3. на следующей строке напиши print("Hello, ", name). Через запятую в команде print() или write() можно выводить несколько значений.

Например, текст и переменную:

```
name=read()  
print("Hello, ", name)_
```

4. Теперь программу можно запустить кнопкой Run. Но в этот раз нам нужен другой способ запуска программы, которым я советую тебе пользоваться в дальнейшем. Сохрани файл кнопкой Save и выйди из файла в командную строку с помощью кнопки Exit. В командной строке напиши название файла, в котором ты писал программу. В нашем случае это prog3. Просто написав название файла и нажав Enter, мы запустим программу, которая написана в этом файле:

```
> prog1  
Technomage  
Hello, Technomage  
>
```

5. После запуска программы ты увидишь мигающий курсор. Это программа ждет, пока ты напишешь текст. Напиши свое имя на английском и нажми Enter. После того, как ты это сделаешь, ты увидишь текст Hello, «твое имя»:

```
> prog3  
Technomage  
Hello, Technomage  
> _
```

## Система 2. Программирование 4. Условный оператор if.

С помощью этой команды можно выполнить часть кода в зависимости от условия. Например, если необходимо проверить пароль пользователя для входа в систему.

Условие: если пользователь ввел правильный пароль, тогда он входит в систему и ему выводится сообщение об успешном входе. Иначе программа не пропускает пользователя, и ему выводится сообщение о вводе неверного пароля. Конструкция условного оператора выглядит так:

if {условие} then

(команды, если условие верно)

else

(команды, если условие неверно)

end

Например, напомним программу, которая получает от пользователя число и сравнивает его с числом 10. Если число от пользователя больше 10, тогда выводится сообщение о том, что число пользователя больше 10. Иначе выводится сообщение о том, что число пользователя меньше либо равно 10.

1. Создай файл prog4;
2. возьми из сундука страницу «Пример 3»:

```
num=read()
num=tonumber(num)

if num>10 then
    print(num, ">10")
else
    print(num, "<=10")
end
```

3. во 2-ой строке ты увидишь незнакомую команду – `tonumber()`. Эта команда преобразовывает строку в число. Например, строку «15» в число «15».

На 3-ей строке начинается условный оператор `if`. `num>10` – это условие или условное выражение. Условные выражения могут быть `true` (правда) или `false` (ложь). Например, логическое выражение `8>10` будет `false` (ложным);

4. перепиши код:

```
num=read()
num=tonumber(num)

if num>10 then
  print(num, ">10")
else
  print(num, "<=10")
end_
```

5. запусти программу из командной строки;

6. протестируй выполнение программы, вводя числа меньше 10, число 10, числа больше 10:

```
> prog4
5
5<=10
> prog4
10
10<=10
> prog4
15
15>10
> _
```

Условных выражений может быть несколько. Например, если (`if`) ответ ученика равен 10 ИЛИ (`or`) 20, тогда (`then`) выводится сообщение о том, что ученик ответил правильно. Иначе (`else`) выводится сообщение о том,



что ученик ответил неправильно. В коде это будет выглядеть так:

```
if {answer==10 or answer==20} then
```

(команды, если условие верно)

```
else
```

(команды, если условие неверно)

```
end
```

Команды после слова then выполняются, если будет истинно условное выражение после слова if. Условное выражение будет истинно, если ответ будет равен 10 ИЛИ 20.

Еще пример. Если (if) введенный пользователем логин равен "admin" И (and) пароль равен «7845», тогда (then) пользователь входит в систему. Иначе (else) выводится сообщение о том, что пользователь ввел неверный логин или пароль.

В коде это будет выглядеть так:

```
if {login=="admin" and password=="7845"} then
```

(команды, если условие верно)

```
else
```

(команды, если условие неверно)

```
end
```

Команды после слова then выполняются, если будет истинно условное выражение после слова if. Условное выражение будет истинно, если логин равен «admin» И пароль равен «7845».

Для сравнения значений в условных выражениях используются знаки:

1. > (больше). Например,  $x > 10$ ;
2. < (меньше). Например,  $x < 20$ ;
3. >= (больше либо равно). Например,  $x \geq 10$ ;
4. <= (меньше либо равно). Например,  $x \leq 10$ ;
5. == (равно). Например,  $x == 15$ ;
6. ~= (не равно). Например,  $x \neq 15$ .

## Система 2. Инструкция.

Отлично! Ты усвоил все необходимое, чтобы запрограммировать систему входа. Вот тебе инструкция по написанию этой программы:

1. создай в корневом каталоге файл под названием loginSystem;
2. создай 2 переменные username (имя пользователя) и password (пароль). В качестве значений прими ввод от пользователя. Перед вводом выведи сообщение о том, что пользователь должен ввести имя пользователя и пароль;
3. с помощью условной конструкции if проверь имя пользователя И пароль. В качестве имени будет значение строка «admin», а в качестве пароля – строка «894324»;
4. в случае если пользователь ввел правильные имя пользователя и пароль, то выведи сообщение о том, что пользователь вошел в систему;
5. иначе выведи сообщение о том, что пользователь ввел неправильный пароль или имя пользователя:

```
write("Username: ")
username=read()
write("Password: ")
password=read()

if username=="admin" and password=="894324" then
  print("You are logged in system")
else
  print("Incorrect password or username")
end
```

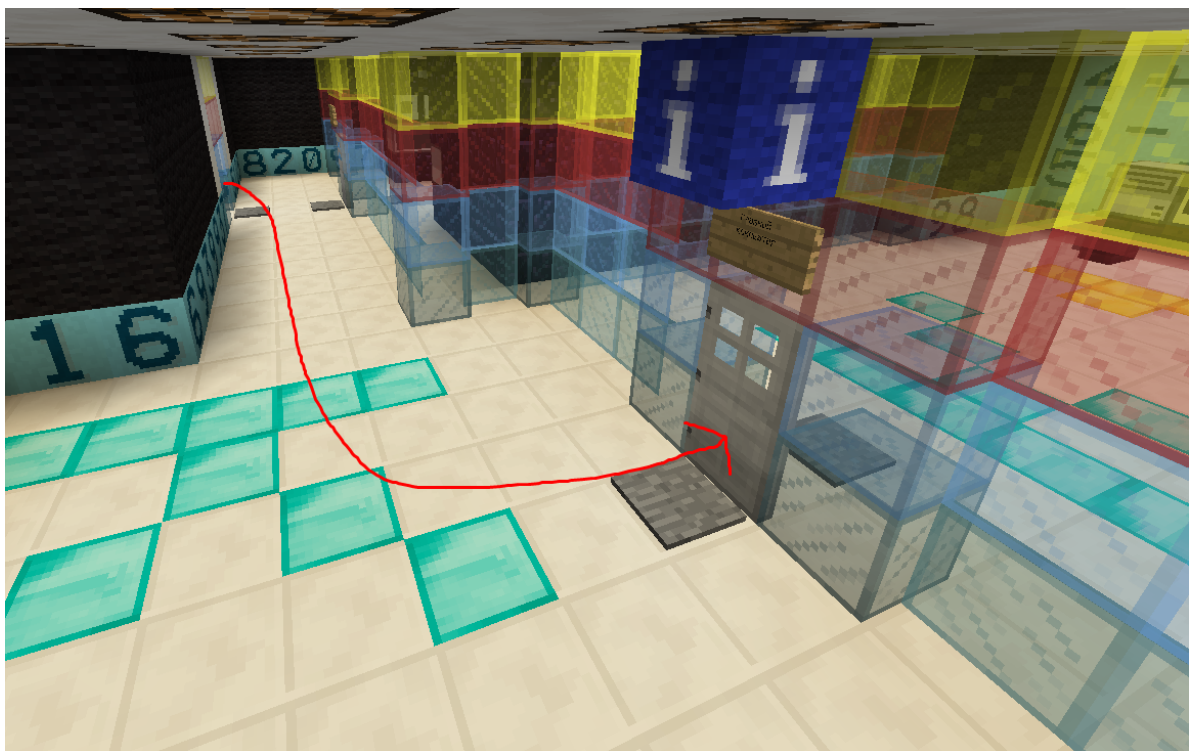
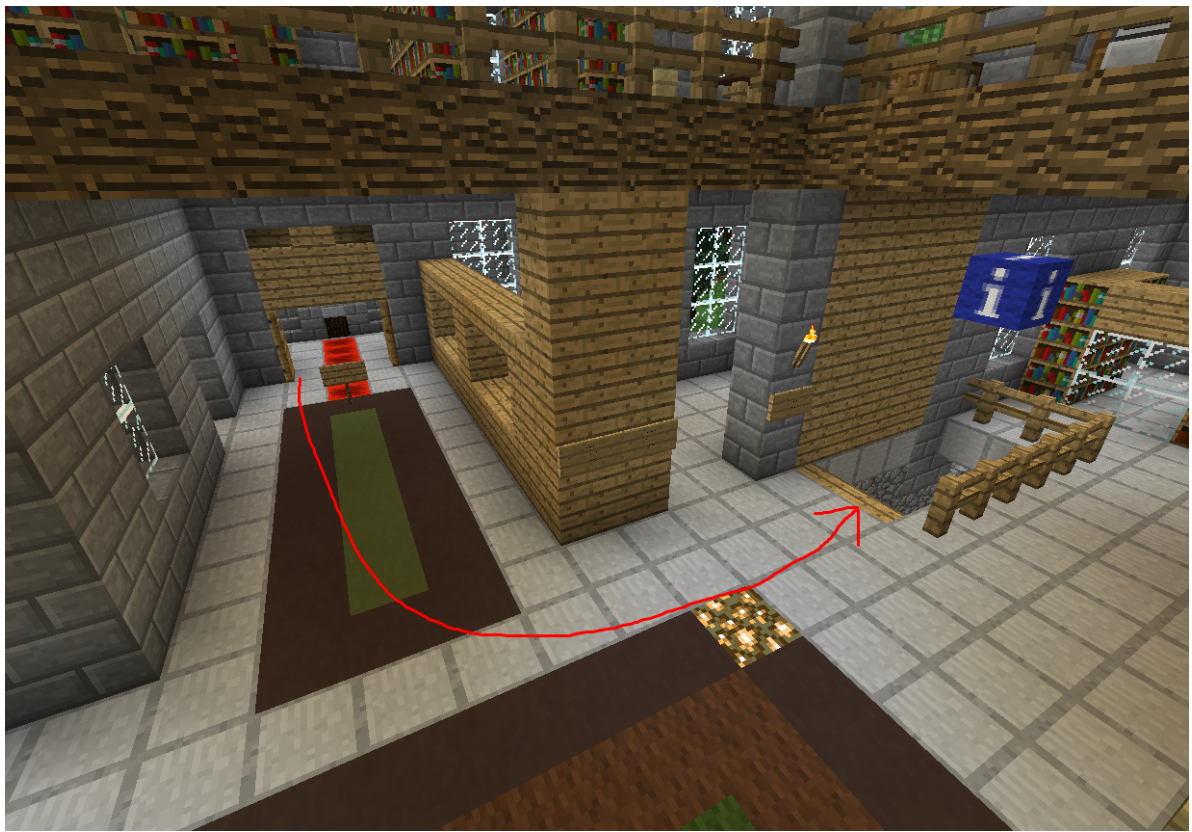
6. после того, как ты запрограммировал систему, проверь ее на работоспособность. Сначала попробуй ввести правильные логин и

пароль, затем правильный логин и неправильный пароль, затем неправильный логин и правильный пароль и, наконец, неправильный логин и неправильный пароль:

```
> loginSystem
Username: admin
Password: 894324
You are logged in system
> loginSystem
Username: admin
Password: 594324
Incorrect password or username
> loginSystem
Username: administrator
Password: 894324
Incorrect password or username
> loginSystem
Username: maksim
Password: 784376
Incorrect password or username
> _
```

Поздравляем! Вы написали программу входа в систему!

**Система 3. Задание:** Отправляйся отсюда налево в серверную под 1-ым этажом. Найди помещение под названием «Главный компьютер». Зайди туда и ты увидишь компьютер и сундук с инструкциями, которые подскажут тебе, что делать:





Для восстановления программ тебе понадобятся книги «Сумматор» и «Квадратор» из сундука.

### Система 3. Сумматор

Чтобы написать программу «Сумматор», тебе необходимо познакомиться с циклами.

Представь, что тебе необходимо вывести сообщение «Привет!» 50 раз. Если написать код без цикла, то программа окажется довольно длинной. А если необходимо будет поменять текст сообщения, тогда придется менять его во всех 50 командах.

Цикл позволяет решить эти проблемы. Цикл выполняет команды столько раз, сколько это необходимо. Например, так будет выглядеть программа, которая выведет 50 раз сообщение «Hello!»:

```
for i = 1, 50, 1 do
```

```
    print("Hello!")
```

```
end
```

Всего 3 строки кода и 50 сообщений готовы. Если необходимо вывести сообщение 100 раз, тогда достаточно поменять 2-ое число. А если необходимо поменять текст сообщения, то это можно сделать всего 1 раз.

Конструкция цикла for выглядит так:

```
for i = [начальное число], [конечное число], [шаг] do
```

```
    (команды в цикле)
```

```
end
```

Переменная *i* – это счетчик. Он имеет начальное и конечное значения, а также шаг, с которым значения меняются. Например, *i* = 1, 15, 1. Такой цикл выполнится 15 раз. Или *i* = 10, 20, 2. Такой цикл выполнится 5 раз.



## Инструкция по написанию программы «Сумматор»:

1. создай в корневом каталоге каталог под названием progs:



```
CraftOS 1.7
> mkdir progs
> dir
progs rom
> cd progs
progs>
```

2. создай файл summator в каталоге progs. В этом файле ты будешь писать код;

3. создай 2 переменные. В одной будет храниться сумма чисел. Изначально она должна равняться нулю. Переменная должна создаваться перед циклом. Вторая переменная нужна для получения от пользователя числа, до которого включительно будет прибавляться сумма.

Не забудь, что команда `read()` получает от пользователя строку. Чтобы преобразовать строку в число, необходимо на следующей строке после команды `read()` воспользоваться командой `tonumber()`. Например:

```
x=read()
```

```
x=tonumber(x);
```

4. далее создай цикл, у счетчика которого начальное значение будет равно 1, конечное – числу, введенному пользователем, а шаг равен 1;

5. в цикле прибавляй к переменной, в которой хранится сумма, значение счетчика. Сделать это можно так: `x=x+i`;

6. после цикла выведи сумму на экран:

```
num=read()
num=tonumber(num)

sum=0
for i = 1, num, 1 do
    sum=sum+i
end

print(sum)_
```

7. протестируй программу. Например, при вводе числа 3 должна выводиться сумма, равная 6. При вводе числа 5 должна выводиться сумма, равная 15. При вводе числа 7 должна выводиться сумма, равная 28:

```
progs> summator
3
6
progs> summator
5
15
progs> summator
7
28
progs> _
```

### Система 3. Квадратор

Чтобы написать программу «Квадратор», тебе необходимо познакомиться еще с одним циклом. Он называется `while..do`. В отличие от цикла `for` у него есть условное выражение. Если условие выполняется, то цикл `while..do` продолжается. Если условие перестает выполняться, то цикл прекращается.

Например, программа считает количество шариков. Ей необходимо досчитать до 25. В цикле каждый раз прибавляется единица к переменной, в которой хранится количество шариков. Цикл будет выполняться до тех пор, пока количество посчитанных шариков в переменной не будет равно 25. В коде это будет выглядеть так:

```
sum=0
```

```
while sum<=25 do
```

```
    sum=sum+1
```

```
end
```

Конструкция цикла:

```
while {условие} do
```

```
    (команды в цикле)
```

```
end
```

Инструкция по написанию программы «Квадратор»:

1. создай файл `quadrator` в каталоге `progrs`. В этом файле ты будешь писать код;
2. создай переменную и сохрани в нее число, полученное от пользователя. Это число, до которого включительно будут выводиться квадраты;
3. создай переменную-счетчик `i`, равной 1;
4. создай цикл `while..do`. Условие должно быть следующим: цикл выполняется, пока счетчик меньше либо равен числу, которое ввел пользователь;
5. в цикле умножь значение счетчика само на себя и выведи его на экран. Это можно сделать в одной строке с помощью такой команды – `print(i*i)`. Сначала счетчик умножается на себя, а затем результат выводится на экран;
6. в цикле прибавь к счетчику единицу:

```
num=read()
num=tonumber(num)

i=1
while i<=num do
  print(i*i)
  i=i+1
end_
```

7. протестируй программу. Например, при вводе числа 4 должны выводиться числа 1 4 9 16. При вводе числа 6 должны выводиться числа 1 4 9 16 25 36:

```
progs> quadrator
4
1
4
9
16
progs> quadrator
6
1
4
9
16
25
36
progs> _
```

После выполнения последнего задания ученик возвращается в кабинет Техномага и отмечает выполненное задание. Третий рычаг активирует механизм, который дает ученику книгу с благодарностью и подарок от Техномага:



### **Подведение итогов – 5 мин.**

Сегодня ученики узнали много нового и освоили навыки настоящего программирования.

*Спросите у ребят, что нового они узнали на сегодняшнем занятии, и остановитесь на основных моментах пройденного материала.*